

Training a reinforcement learning controller for a fixed set of traffic lights

Technical report. Experiment [exp_20260105_090955](#); analysis and data in [paper_rl_vs_tm/](#).

Summary

We trained a reinforcement learning agent to set the signal timings of one junction set — a 34-junction grid — and compared it against Tree Method, a decentralized adaptive controller, on that junction set's traffic scenario. The agent improves throughput by 5.7% (8,660 against 8,194 veh/h) and cuts average trip duration by 20.6% (577.5 s against 727.0 s). Over ten further simulation seeds, which vary driver behaviour while holding demand fixed, it produces shorter trips on all ten, by 137.6 s on average (−17.8%, 95% CI 88.3–187.0 s).

The improvement is a genuine speed-up rather than an artefact of which vehicles finish: 91% of the gap comes from the 16,132 vehicles that arrive in both runs travelling faster, mostly by waiting less.

Analysing the trained policy shows that it settles on a near-constant timing plan for each junction — one phase dominant at 18 junctions, a near-equal split at the rest. That plan, extracted from the agent's own outputs and played without the model, carries the whole gain. The plan is a product of the training: measured on an earlier simulator version, a generic equal-split plan performs far worse than Tree Method.

Compute limited the work to one scenario. Training the same junction set across demand levels, departure patterns and many seeds is the natural continuation, not a change of method.

1. The problem

Traffic signal control governs urban mobility: poorly timed signals produce congestion and long travel times. Improving throughput and reducing waiting requires coordinated decisions across intersections, because congestion at one junction propagates downstream.

Reinforcement learning is a natural fit for adaptive control under changing conditions, but at network scale it raises its own problems. The reward structure determines which coordination behaviours the agent learns and how well network-wide performance can be attributed to individual signal decisions. Training is expensive, since every sample requires a traffic simulation.

This report answers a narrow, practical question: **can an agent trained for a specific set of traffic lights outperform an adaptive controller on that junction set?** The baseline is Tree Method, a decentralized bottleneck-based controller already implemented in the same codebase and operating on the same signals.

2. Formulation choices

Building an RL signal controller requires seven decisions. Each was taken with the proof-of-concept goal in mind: reach a working controller on one junction set quickly, rather than a general one slowly.

Axis	Choice	Reason
Training scope	Network-specific	The target is a fixed junction set. A network-agnostic model needs graph or attention architectures and far more training.
Agent architecture	Centralized	One agent sees the whole network and coordinates directly, with no credit-assignment problem between agents. Feasible at 34 junctions.
Input model	Macroscopic	Per-edge aggregates are compact and stable, and match the granularity at which signals operate. Microscopic state scales poorly with traffic volume.
Reward design	Statistical, global, intermediate	Network-wide aggregates computed every step: dense feedback, low variance, no per-vehicle tracking overhead.
Action representation	Phase durations	The agent sets the green duration of each phase (see §3.2).
Time resolution	Fixed interval, one 90 s cycle	Stable decisions, aligned with how signal plans are actually set.
Exploration	Constrained	A 10 s minimum green is enforced by the environment, so every policy explored is deployable.

The trade-offs behind each axis are set out in full in [docs/RL_DISCUSSION.md](#) §2. Two choices departed from the initial recommendation during implementation: the action space became continuous phase durations rather than discrete phase selection (§3.2), and decisions are taken once per cycle rather than every 10 s.

3. Environment

The environment wraps the existing SUMO simulation infrastructure behind a Gymnasium interface, and reuses Tree Method's traffic analysis rather than reimplementing it.

3.1 State

Each edge contributes nine features. Four are elementary: speed ratio against free flow, vehicle density, flow rate, and a congestion flag raised when waiting exceeds 30 s. Four are computed with Tree Method's own [Link](#) and [QmaxProperties](#) classes: normalized density via [calc_k_by_u](#), theoretical flow, a bottleneck flag ([speed < q_max_u](#)) and normalized time loss. The ninth is a speed trend from a moving average.

Each junction contributes four features — normalized phase, duration, incoming and outgoing flow — and six network-level features summarize the state: bottleneck ratio, normalized cost, vehicle count, average speed, congestion ratio and cycle length.

The alignment with Tree Method is deliberate. The agent sees traffic the way the baseline sees it, so the comparison tests the controller rather than the instrumentation.

3.2 Actions

Both controllers decide the same quantity: for each junction, the green duration of each phase in the next 90-second cycle. Phases run in a fixed order and each gets at least 10 s.

The 28 four-phase junctions use the `partial_opposites` phasing — NS straight+right, NS left+U-turn, EW straight+right, EW left+U-turn — and the 6 boundary junctions (A0, A2, A4, A5, F0, F5) have three phases. Opposing directions always share a phase.

The policy emits 136 values (34 junctions $\times$ 4) in $[-10, +10]$. A softmax turns each junction's four values into shares of the 50 s that remain once the four 10 s minima are met. Tree Method, by contrast, moves each phase by at most a cost-dependent step per cycle.

The initial design used discrete phase selection with fixed 10 s durations. Continuous durations replaced it to give the agent finer control over timing. The bound on those outputs shapes the timings the agent settles on (§7.2).

3.3 Reward

Six components are computed every simulation step from network-wide aggregates:

Component	Formula
Throughput	$+50.0 \times \text{vehicles_completed}$
Waiting time	$-0.45 \times \text{z_score}(\text{avg_waiting_time})$
Excessive waiting	$-2.0 \times \text{count}(\text{vehicles waiting} > 5 \text{ min})$
Speed	$+10.0 \times (\text{avg_speed} / 50.0)$
Bottleneck	$-4.0 \times \text{bottleneck_count}$
Insertion bonus	$+1.0$ if $\text{waiting_to_insert} < 50$

Waiting time is z-score normalized with empirically derived statistics (mean 38.01 s, sd 11.12 s). The coefficients were tuned so that no component dominates: on a full 22,000-vehicle run the waiting penalty accounts for 58% of total magnitude, bottlenecks 31%, speed 14% and throughput 9%, with the reward correlating +0.684 with average speed and -0.708 with bottleneck count.

4. Training

Algorithm. PPO from Stable-Baselines3, on PyTorch, with TraCI driving SUMO. An actor-critic method suits the problem: the critic evaluates traffic states while the actor sets timings, and the value baseline reduces the variance that a global reward would otherwise carry.

Configuration. The configuration below is where five rounds of tuning ended up, each round after the first fixing a specific failure:

```

Learning rate: 1e-4 (scheduled decay to 5e-6)
Clip range: 0.2
Batch size: 2048
N_steps: 4096
N_epochs: 5

```

Gamma:	0.995
GAE lambda:	0.98
Gradient clip:	0.5
Entropy:	0.02 → 0.001 over 500k steps
Early stopping:	10 evaluations patience

The revisions, in order:

1. **Initial** (lr 2e-4, clip 0.1, 2048 steps, 10 epochs, gamma 0.99): stable but slow, and weak on long-horizon coordination.
2. **Long horizon**: gamma to 0.995, GAE lambda to 0.98, 4096 steps per update, to capture effects that cascade across the network over minutes rather than seconds.
3. **Imitation learning**: pure RL from scratch was too sample-inefficient given the cost of a simulation. Expert demonstrations were collected from Tree Method and the policy pretrained by behavioural cloning before PPO fine-tuning.
4. **Stability**: a clip range of 0.05, adopted to protect the pretrained policy, caused KL divergence spikes and policy collapse. Standard 0.2 fixed it; epochs dropped from 15 to 5.
5. **Learning rate**: 3e-4 to 1e-4.

Reward coefficient. The throughput coefficient was raised from 0.1 to 0.4 to make throughput the dominant signal. Training became unstable immediately: at 8,192 steps `approx_kl` reached 269.6 against an expected value below 0.1, 90% of updates were clipped, explained variance sat at 0.01, and evaluation peaked at 16k steps and then degraded. The intermediate value 0.2 was used instead, and is the setting in the final experiment; the default 0.1 collapsed in this configuration.

The final run. The policy was pretrained by behavioural cloning on Tree Method demonstrations, then trained with PPO to about 2.47M steps on the TAU cluster, with a checkpoint every 4,096 steps.

Checkpoint selection. Checkpoints from 524,288 to 2,473,984 steps were each run once through the full scenario: 475 rows, one failed run, 474 results. Checkpoint **1,945,600** gave the highest throughput (8,660 veh/h) and the second-lowest average duration (577.5 s; the lowest, 573.2 s, came with lower throughput at checkpoint 2,260,992). It is the model used throughout the rest of this report.

5. Experimental setup

Network. A 6×6 orthogonal grid with two junctions removed (A3 and B3), leaving 34 signalized junctions: 280 m blocks, realistic lane counts, `partial_opposites` phasing, network seed 24208.

Demand. 22,000 passenger vehicles on inner routes, uniform departures, starting at 08:00 and running 7,300 s (about two hours), with `realtime` routing; traffic seeds 72632 (private) and 27031 (public).

Reference environment. TAU cluster, SUMO 1.25.0, default SUMO seed 23423. The pinned environment matters: the same code and model on SUMO 1.22.0 give materially different numbers (§8). Ten further SUMO seeds vary driver behaviour only — network, demand, routes and departure times are identical across them.

Reproducibility. The experiment is self-contained in `paper_rl_vs_tm/`: a frozen copy of the application, the model and its selection data, pinned requirements, the run scripts and every result file. Each number below is traceable to a file in `results/` through `FINDINGS.md`.

6. Results

6.1 The agent outperforms the adaptive baseline

On the reference seed:

	Throughput (veh/h)	Average trip duration (s)
Tree Method	8,194	727.0
RL (checkpoint 1,945,600)	8,660	577.5
Difference	+5.7%	−20.6%

Over ten further SUMO seeds:

	Average duration: mean (sd)	Throughput: mean (sd)
Tree Method	775.4 s (54.7)	7,865 (382)
RL	637.8 s (30.9)	8,378 (137)

Paired across those ten seeds, RL's trips are shorter by 137.6 s (−17.8%, 95% CI 88.3–187.0 s) and shorter on 10 of 10; throughput is higher by 513 veh/h (CI 217–809) and higher on 9 of 10. The reference seed is RL's best of the eleven runs, so the −20.6% headline is at the favourable end and −17.8% is the typical gap. Tree Method also varies most across seeds (sd 54.7 s against RL's 30.9 s), so the agent is both faster and steadier.

6.2 The gain is a real speed-up

Average trip duration counts only vehicles that arrive, so a controller could in principle "improve" it by finishing a different, easier set of vehicles. It does not. On the reference seed 16,132 vehicles arrive in both runs, and the 149.4 s gap splits cleanly:

Term	Seconds	Share
The same vehicles travelling faster	136.0	91%
Composition — which vehicles arrived	13.4	9%

For those matched vehicles, waiting time falls from 465.6 s to 371.4 s (−94.2 s) and time loss from 577.9 s to 452.0 s. Routes are 8.4% shorter (1,613 m to 1,478 m) with fewer reroutes (4.7 to 3.6). 67.2% of matched vehicles are faster and 32.3% slower; the winners gain 3.66M vehicle-seconds against the losers' 1.47M.

The advantage grows with congestion: 21 s for vehicles departing in the first 15 minutes, rising to 212–239 s for those departing between 3,600 s and 5,400 s.

6.3 Where the time is saved

This section is descriptive. Only the plan as a whole was tested causally (§7.3); the splits below show where the saved time shows up, not what caused it. Means over all 11 runs:

Mean over 11 runs	Tree Method	RL
Waiting (veh-h)	2,878	2,366
Time loss (veh-h)	3,483	2,831
Blocked tail lane-intervals	2,013	1,651

Waiting is lower under RL in 11 of 11 runs. The two controllers are level for the first 15 minutes (101 against 100 veh-h); Tree Method then accumulates delay faster, reaching 495 against 378 veh-h in the last full 15 minutes. RL has *more* blocked queues than Tree Method in the first 45 minutes and fewer thereafter.

Splitting each matched vehicle's trip time across the junction approaches it used, vehicles save 518 veh-h in total (122.8 s each, positive in 11 of 11 seeds):

- **Gains lie in the centre and east.** 20 of 34 junctions save time in at least 10 of 11 seeds; the top eight (D4, D3, C3, C2, D2, E3, E4, C1) account for 349.3 veh-h, 56% of the time saved at gaining junctions.
- **Losses lie in the west:** B4 (−43.6 veh-h), B2 (−21.5), B5 (−20.1), A4 (−16.6), A2 (−12.6) and A1 (−8.2), together with C5, total 122.7 veh-h. Four of these neighbour the two removed junctions, where the grid is irregular.
- At the nine junctions whose plan favours NS straight+right, 189.3 of the 204.6 veh-h they save (92%) is on NS approaches.

7. What the agent learned

7.1 A near-constant plan per junction

Measured from the signals actually displayed, over 81 cycles:

Per junction	Tree Method	RL
Phase greens unchanged from previous cycle	35.1%	85.7%
Mean cycle-to-cycle change of a phase's green	2.0 s	0.3 s

After two or three cycles each junction holds an almost fixed plan, of one of two kinds. At 18 junctions one phase dominates (40 s or more) while the others sit at or near the 10 s minimum. Twelve of these run 45–60 s on one straight+right phase — NS at B5, C2, C3, C4, C5, D2, E3, F2 and F3, and EW at D5, E4 and E5 — and the remaining six are the three-phase boundary junctions, where the controller folds the unused fourth duration into phase 2; counted on the model's raw outputs instead of the displayed signals, 16 junctions have a dominant phase. The other 16 junctions hold a near-equal split — 15 at 24/22/22/22 and D1 at 21/21/27/21.

Overall the agent gives NS 55.9% of green time against Tree Method's 51.1%, holds one of the three smaller phase types at the 10 s minimum far more often (22–26% of cycles against 5–12%), and gives NS straight+right 60 s or more in 18.7% of cycles against Tree Method's 0.3%. It commits where Tree Method hedges.

7.2 Why the plan is stable

Logging all 136 outputs at each of the 82 decisions shows where the stability comes from. Most outputs fall below the lower bound of the action space and are clipped to it: a median of 115 of 136 per decision, with raw values reaching -190.5 . No output ever exceeds $+10$. From $t = 270$ s the median standard deviation of an output over the whole run is 0.58 , and only 13 of the 136 ever cross the bound. Of the 19 outputs never clipped, 16 are exactly the dominant phases of the 16 junctions whose plan has one. The plan follows arithmetically from the clipping: four outputs at the bound produce an equal split, and one output above it takes nearly all the spare time.

The mechanism is visible in the saved model. The policy is Gaussian with clipping rather than squashing, and its exploration standard deviation is 1.19 – 1.32 per output. Clipped outputs sit a median of five standard deviations below the bound, so during training the sampled actions were almost always clipped, the environment responded identically whatever their value, and no gradient pushed them back into range. One plausible origin has not been verified: behavioural cloning encoded missing phases as $\log(1e-8) \approx -18.4$, already below the bound.

7.3 The plan carries the whole gain

Plans built from the agent's own logged outputs and played every cycle, on the reference seed:

Run	Decisions differing from RL (of 2,788)	Throughput	Avg duration
Exact replay of RL's durations	0	8,660	577.5
Modal plan every cycle	459	8,402	629.6
RL's first 3 cycles, then modal plan	391	8,380	632.3
RL's durations at the 10 switching junctions	57	8,445	619.1
Both of the above	19	8,317	642.1

The exact replay reproduces the RL run trip for trip and switch for switch, which validates the mechanism. Every plan-based run lands between 619 s and 642 s, all far below Tree Method's 727.0 s, and over ten seeds the modal plan is shorter than Tree Method on 10 of 10 (-148.6 s, CI 117.4 – 179.8) and statistically indistinguishable from the agent itself ($+10.9$ s, CI -14.8 to $+36.6$).

So the agent's advantage is its junction-specific timing plan, taken as a whole. Two things follow. First, the plan is what the training produced, and it is not a plan anyone had before: in an earlier comparison on the same scenario, SUMO's default equal-split static program reached 5,628 veh/h / 881.7 s against Tree Method's 8,507 / 708.7 s, so a generic fixed plan is far worse than the baseline while the learned one is better. (That comparison ran on SUMO 1.22.0 and is context only, not a measurement in the reference environment.) Second, what the training produced can be written down: 34 sets of four durations, which is what the plan runs above played back in place of the model's decisions.

The stability follows from the action bound (§7.2). Widening or smoothing that bound would let the agent vary its timings within the cycle as well.

8. Limitations

1. **One scenario.** Network, demand and traffic seeds are fixed. This is a compute constraint, not a claim about scope; the SUMO seeds vary only driver behaviour.
2. **Simulator version.** The same code and model give 8,507 / 708.7 s (Tree Method) and 8,504 / 618.4 s (RL) on SUMO 1.22.0. All numbers here are SUMO 1.25.0, and controllers must only ever be compared on one version.
3. **Sensitivity.** Changing 19 decisions by 2–3 s moved average duration by 64.6 s. Single-run differences smaller than roughly this should not be interpreted, which is why the seed results in §6.1 carry the argument rather than the headline run.
4. **§6.3 is descriptive.** Only the plan as a whole was tested causally (§7.3). Attributing the gain to particular junctions would need junction-group swap runs, which were not done: a few chosen groups cannot represent all combinations.
5. **Average duration counts arrived vehicles only.** On the reference seed 2,837 (Tree Method) and 2,165 (RL) vehicles remain in the network at 7,300 s, and 2,548 / 2,274 were never inserted. §6.2 bounds the effect at 9% of the gap.
6. **Three-phase junctions.** At inference the fourth duration is added to phase 2, while the training environment drops it — a discrepancy found in the code and not quantified.
7. **Teleports.** 270 (Tree Method) and 257 (RL) on the reference seed; among matched vehicles their contribution is about zero.

9. Conclusion

An agent trained for one specific set of traffic lights beat an adaptive controller on that junction set: 5.7% more throughput and 20.6% shorter trips on the reference seed, and shorter trips on all ten further seeds, averaging 17.8%. The gain is a real speed-up of the same vehicles, it grows as the network loads, and it survives the change in driver behaviour that the seeds introduce.

What the training produced is a junction-specific timing plan that outperforms a controller adapting in real time. That plan came out of the RL search.

Two continuations follow. The first is scale: train the same junction set across demand levels, departure patterns, times of day and many seeds, which is what a deployment would require and what compute prevented here. The second is the action bound: it holds the agent's outputs at their limits, which is what fixes the plan in place (§7.2).